Rapport de mi-parcours

Accenture Strategy & Consulting

Création d'agents basés sur modèles de langue pour les services financiers

Groupe :
BENYAHIA Mathis
FORTUNY Arnaud
PORET Guillaume

5 janvier 2026

Table des matières

1 Introduction **1**

2 Approche globale **2**
2.1 Structure du projet . . . 2
2.2 Analyse de concurrence . . . 3
2.2.1 Cartographie des acteurs . . . 3
2.2.2 Tableau comparatif synthétique . . . 3
2.2.3 Forces et faiblesses des concurrents . . . 3
2.2.4 Valeur ajoutée du projet . . . 3
2.3 Retroplanning . . . 3

3 Partie Logicielle **5**
3.1 Cas d'utilisation classique et modèles . . . 5
3.2 Choix techniques . . . 6
3.2.1 Base de donnée . . . 6
3.2.1.1 Choix des serveurs . . . 6
3.2.1.2 Authentification . . . 7
3.2.2 Interface utilisateur . . . 7
3.2.2.1 Page d'authentification . . . 7
3.2.2.2 Page principale . . . 7
3.2.2.3 Interaction avec le backend . . . 8
3.3 Principaux axes d'améliorations . . . 8
3.3.1 Stockage des conversations . . . 8
3.3.2 Base de donnée temporaire . . . 8
3.3.3 Déploiement . . . 8

4 Partie IA **9**
4.1 Etat de l'art . . . 9
4.2 Choix techniques et architecture de l'agent . . . 9
4.2.1 Choix du modèle de langage . . . 9
4.2.2 Orchestration et intégration dans le système . . . 10
4.2.3 Architecture hybride de l'agent . . . 10
4.2.4 Génération de contraintes structurées . . . 10
4.2.5 Flux global de fonctionnement . . . 10
4.3 Pipeline de traitement et garde-fous . . . 10
4.3.1 Pipeline de traitement . . . 10
4.3.2 Mise en place de garde-fous . . . 11
4.4 Exemples d'utilisation . . . 11
4.4.1 Structuration d'une demande en contraintes formelles . . . 11
4.4.2 Structuration d'une contrainte de risque implicite . . . 11
4.4.3 Comparaison quantitative de deux actifs . . . 12

5 Partie Finance **13**
5.1 Introduction . . . 13
5.2 Etat de l'art . . . 13
5.2.1 Modèle d'évaluation des actifs financiers . . . 13
5.2.2 Optimisation sous contraintes . . . 13
5.2.3 Exemple d'optimisation à deux actifs . . . 14
5.3 Architecture . . . 15

5.4 Choix techniques 16
5.4.1 Sources de données 16
5.4.2 ajustement du portefeuille 17
5.5 Problèmes rencontrés 17
5.5.1 Disponibilité d'informations financières 17
5.5.2 Construction de portefeuilles sur des données historiques 17

6 Conclusion **19**

Bibliographie **20**

Annexes **22**

Chapitre 1

Introduction

L'essor récent des modèles de langage de grande taille (Large Language Models, LLM) a profondément transformé de nombreux secteurs, et en particulier celui des services financiers. Grâce à leur capacité à comprendre et générer du langage naturel, ces modèles ouvrent de nouvelles perspectives pour l'assistance à la prise de décision, la démocratisation de l'accès à l'information financière et l'amélioration de l'expérience utilisateur. Banques, sociétés de gestion et institutions financières explorent ainsi de plus en plus l'intégration de solutions basées sur l'intelligence artificielle afin de proposer des services personnalisés et interactifs.

Cependant, l'utilisation directe des LLM généralistes dans un contexte financier soulève plusieurs limites majeures. D'une part, ces modèles produisent des réponses souvent génériques et peu adaptées aux contraintes spécifiques de l'investissement financier. D'autre part, ils ne garantissent ni la fiabilité des données chiffrées ni la reproductibilité des résultats, et peuvent générer des informations erronées, phénomène connu sous le nom d'hallucination. Enfin, leur coût computationnel et énergétique élevé constitue un frein à leur déploiement à grande échelle, notamment pour des applications spécialisées nécessitant des réponses précises et encadrées.

Face à ces enjeux, les institutions financières s'orientent vers des approches hybrides combinant modèles de langage et outils déterministes spécialisés. L'objectif n'est plus de confier la décision financière à un LLM autonome, mais d'utiliser ce dernier comme une interface intelligente capable de structurer les besoins exprimés par l'utilisateur, tout en s'appuyant sur des modèles financiers rigoureux pour les calculs et les décisions d'investissement. Cette séparation des responsabilités permet de tirer parti des capacités linguistiques des LLM tout en conservant la fiabilité et la transparence indispensables au domaine financier.

C'est dans ce contexte que s'inscrit ce projet de fin d'études, réalisé en collaboration avec Accenture Strategy Consulting. L'objectif du projet est de définir et de développer un cadre applicatif permettant la création d'agents basés sur des modèles de langage, dédiés à l'élaboration de stratégies d'investissement personnalisées pour une institution financière fictive. L'entreprise nous a donné une grande marge de manoeuvre quant à l'orientation que l'on souhaitait donner au projet. Nous avons du ainsi définir le cas d'usage de l'outil que nous allions créer afin d'extraire une problématique du sujet.

Comment démocratiser l'accès à une gestion de portefeuille personnalisée, optimisée et transparente pour les investisseurs particuliers, qu'ils soient novices ou experts, tout en combinant la flexibilité du langage naturel avec la rigueur des algorithmes d'optimisation financière avancés ?

Ces agents doivent être capables d'interagir avec l'utilisateur en langage naturel, d'extraire et de formaliser ses contraintes d'investissement, puis de s'appuyer sur des modules financiers déterministes pour construire et optimiser un portefeuille répondant à ces contraintes. Notre cible est clairement un public d'investisseurs particuliers ayant une compréhension financière plus ou moins avancée.

Le projet repose sur une architecture à trois composantes principales : une couche logicielle assurant l'interface utilisateur et la gestion des interactions, une couche d'intelligence artificielle chargée de l'analyse sémantique et de la structuration des requêtes, et une couche financière dédiée à l'évaluation des actifs et à l'optimisation de portefeuilles sous contraintes. Cette organisation vise à garantir à la fois l'ergonomie du système, la robustesse des traitements et la fiabilité des résultats produits.

Ce rapport présente les travaux réalisés à mi-parcours du projet. Il détaille l'approche globale adoptée, les choix techniques effectués pour chacune des composantes, ainsi que les premières implémentations et résultats obtenus. Enfin, il met en évidence les limites rencontrées et les axes d'amélioration envisagés pour la suite du projet, notamment en matière de déploiement, de gestion des données et d'évolution des stratégies d'investissement proposées.

Chapitre 2

Approche globale

2.1 Structure du projet

Le projet se structure en trois branche : logiciel, IA et finance. La partie logicielle est celle qui sera le plus proche de l'utilisateur. L'IA et la finance représentent le coeur du projet. L'IA communiquera avec le logiciel pour récupérer les besoins de l'utilisateur et transformer en texte les conseils de la partie finance. Cette dernière branche va se servir de besoins extrait par l'IA pour générer des conseils personnalisés.

On obtient ainsi une structure à trois couches résumée par le schéma 2.1

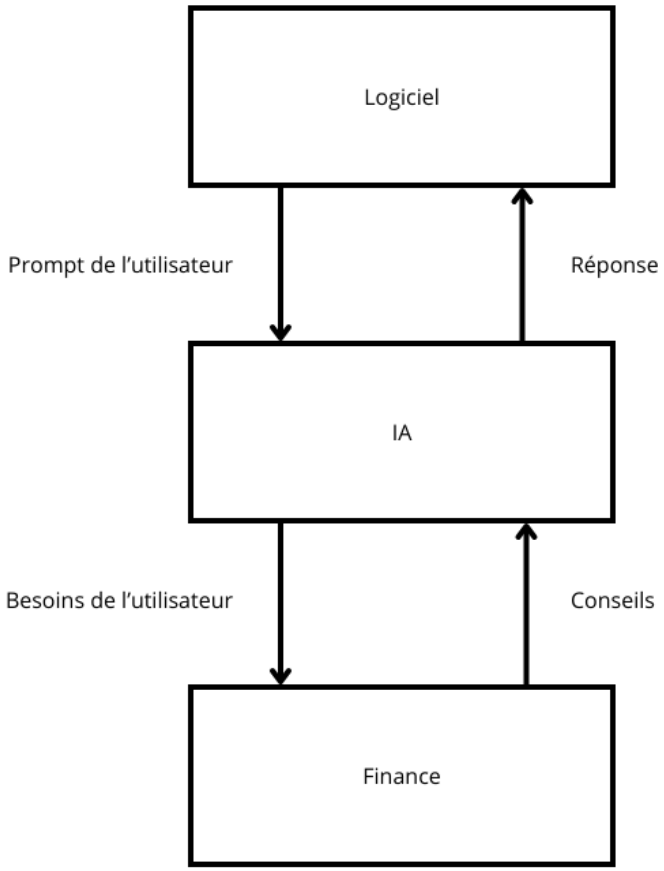


FIGURE 2.1 – Schéma de fonctionnement

2.2 Analyse de concurrence

2.2.1 Cartographie des acteurs

Le marché des outils d'investissement personnalisés pour les particuliers est dominé par deux types d'acteurs :

- **Robo-advisors traditionnels** (Betterment, Wealthfront, Yomoni, Nalo) : Utilisent des questionnaires standardisés et une optimisation algorithmique classique (ex. : Mean-Variance). Leur force réside dans leur simplicité, leur intégration avec les *brokers*, et leur modèle économique éprouvé (frais de gestion 0,25% AUM). Leurs faiblesses incluent une personnalisation limitée, des questionnaires rigides, et un manque de gestion avancée des contraintes (ex. : exclusions sectorielles, critères ESG).
- **Outils « AI-first »** (Quantum Street AI, DeepSeek, Trade Ideas) : Exploitent le NLP et l'IA pour générer des recommandations, mais ciblent principalement les professionnels. Leur complexité et leur coût élevé limitent leur adoption par le grand public.
- **Plateformes hybrides** (Revolut Wealth, eToro Smart Portfolios) : Intègrent des fonctionnalités d'IA basique et des interfaces conversationnelles, mais restent limitées en personnalisation et en gestion des contraintes avancées.

2.2.2 Tableau comparatif synthétique

Table 2.1 – Comparaison des principaux concurrents

Acteur	**Technologie**	**Personnalisation**	**Points forts**	**Points faibles**
Betterment / Wealthfront	Optimisation classique	Questionnaire	Réputation, intégration avec des *brokers*	Questionnaire rigide
StockHero / Trade Ideas	IA générative, bots	Niveau expert	Flexibilité, analyses en temps réel	Complexité, coût élevé
Danelfin	Scoring d'actions par IA	Intermédiaire	Prix abordable, transparence	Base d'utilisateurs limitée

2.2.3 Forces et faiblesses des concurrents

- **Forces** :
  - Robo-advisors : Simplicité, accessibilité, modèle économique clair.
  - Outils AI-first : Analyse prédictive avancée, personnalisation pour les experts.
- **Faiblesses** :
  - Manque de personnalisation fine et d'interfaces en langage naturel.
  - Modèles tarifaires opaques ou coûteux (ex. : abonnements à 100+$/mois).
  - Peu d'outils combinent guidage pédagogique pour novices **et** contrôle expert.

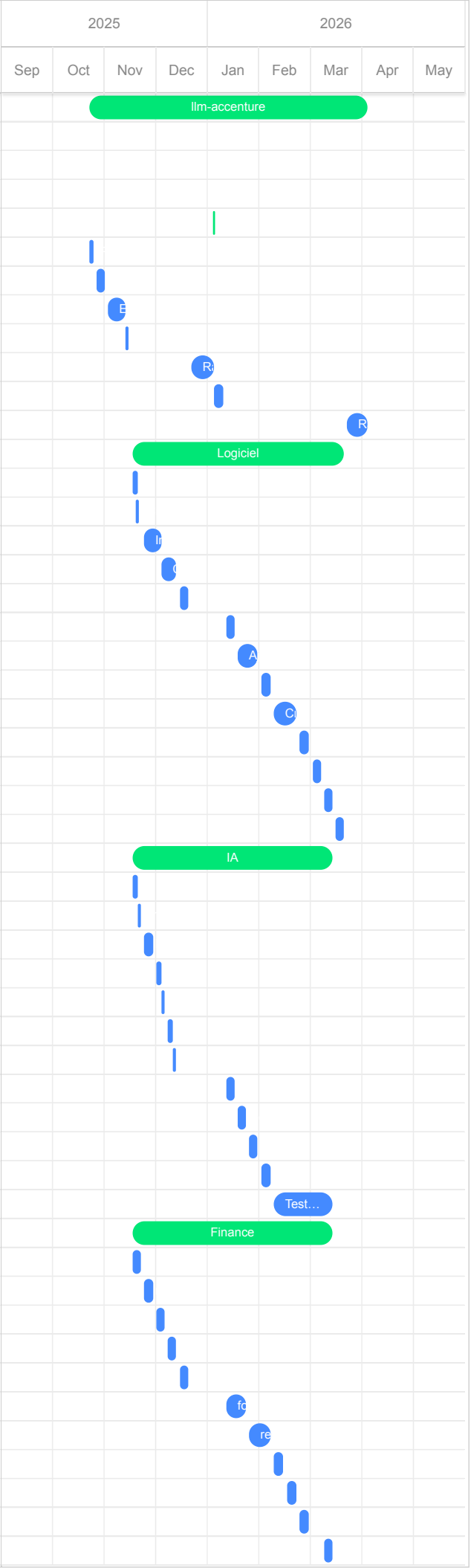
2.2.4 Valeur ajoutée du projet

Notre solution se différencie par :

- Une **interface en langage naturel** pour une interaction intuitive, adaptée aux novices comme aux experts.
- Une **optimisation dynamique des portefeuilles** avec gestion avancée des contraintes (ex. : ratios risque/rendement, ESG).
- Un **double mode d'utilisation** : guidage pédagogique pour les débutants + outils avancés pour les utilisateurs expérimentés.

2.3 Retroplanning

Task Name	Start Time	Duration
Ilm-accenture	2025-10-23	163
To-do		
Doing		
Done		
Global		
Prise de contact avec Accenture		
Compréhension des besoins		
Etat de l'art		
Répartition du travail		
Rapport		
Soutenance		
Rapport final		
Logiciel		
Choix des serveurs pour la base de données		
Création du système d'authentification		
Interface Graphique : page d'authentification et page prir		
Création des routes entre le frontend et le backend		
Intégration de l'IA		
Cryptage des conversation		
Ajout des menus à l'interface		
Création des droits administrateurs		
Création de l'interface admin		
amélioration du hachage et du cryptage		
Intégration de la partie finance		
Mise du backend sur un serveur		
Déploiement		
IA		
Conception de l'architecture IA hybride (LLM + outils Pyt		
Intégration du LLM Mistral (config, prompts, température		
Implémentation du screener IA (langage naturel → JSON		
Validation stricte et garde-fous anti-hallucination		
Interprétation de contraintes qualitatives (ex : volatilité →		
Outils financiers déterministes (prix, stats, comparaison)		
Tests fonctionnels complets et exemples reproductibles		
Intégration avec le module finance (Mathis)		
Gestion de la persistance (historique et profils utilisateur:		
Normalisation et enrichissement des contraintes IA		
Tests d'intégration bout-en-bout (IA → Finance → Résult		
Tests et amélioration de l'IA		
Finance		
recherches documentaires sur les sources de données fi		
recherches documentaire sur optimisation de portefeuille		
recherche sur les classes d'actifs que l'on peu traiter		
construction d'un module de sélection d'univers d'investi		
construction de plusieurs optimiseurs de portefeuilles		
fonctionnalité de mise à jour des portefeuilles		
recherches et implémentations de nouvelles classes d'ac		
mise en commun et intégration dans le reste de la platef		
New taskajout d'une fonctionnalité de gestion des risque		
intégration d'une nouvelle source de données (alpha var		
Test et amélioration de l'optimisations		



This document is created with dhtmlx library: http://dhtmlx.com

Chapitre 3

Partie Logicielle

L'objectif de ce projet est de fournir un produit finis, si les aspects IA et finance en sont le coeur, il y a aussi tout une couche de logiciel à rajouter pour rendre nos conversations simples et utilisables.

Cette partie traite des différents choix effectués pour la base de donnée et l'interface utilisateur ainsi que les axes d'améliorations principaux pour la suite.

3.1 Cas d'utilisation classique et modèles

Le cas d'utilisation que nous avons défini comme "classique" est le suivant : l'utilisateur se connecte, parle avec l'IA et se déconnecte à la fin de sa conversation. Les modèles des autres cas d'utilisations sont en annexe annexe

Nous distinguons deux types de schémas : ceux représentant les interactions entre l'utilisateur et le logiciel, et les diagrammes de séquences.

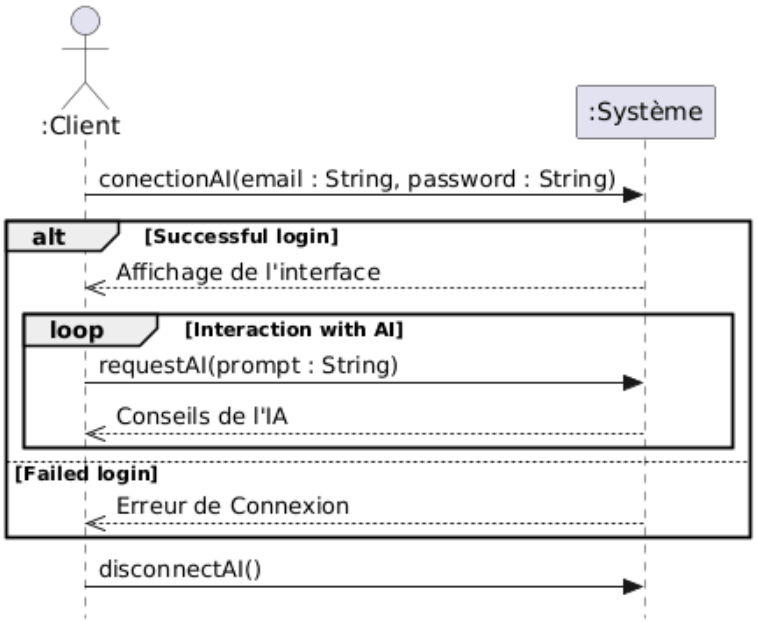


FIGURE 3.1 – Cas d'utilisation classique

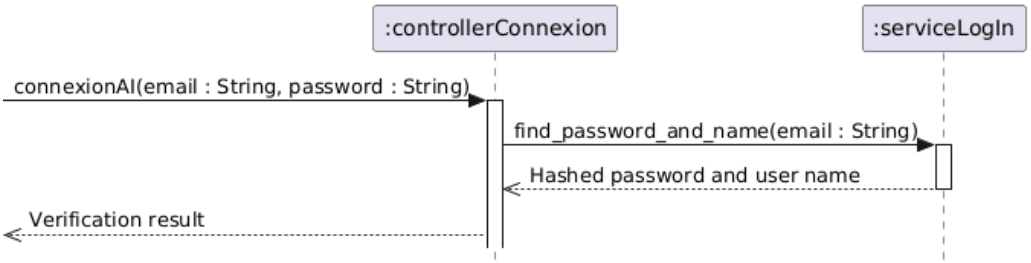


FIGURE 3.2 – Diagramme de séquence : connexionAI

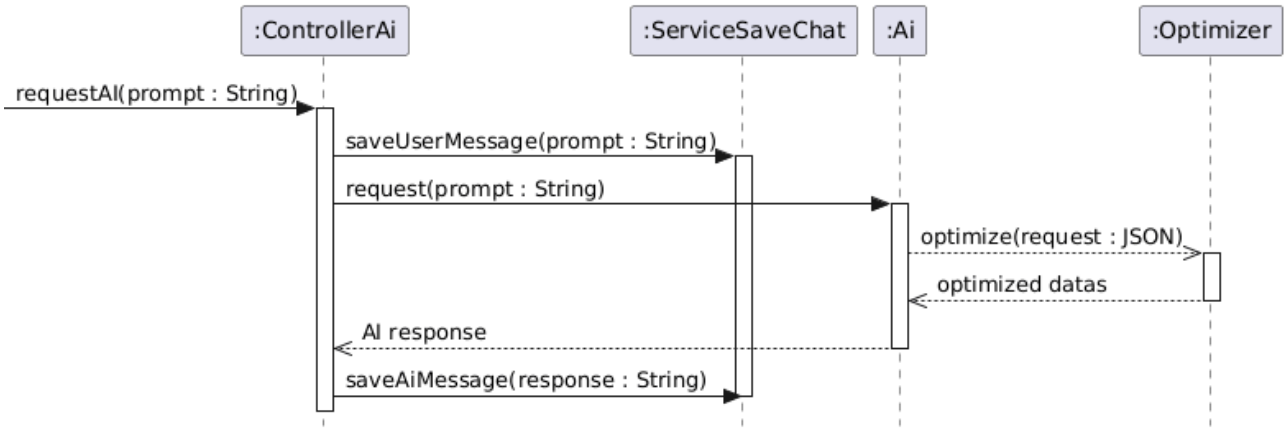


FIGURE 3.3 – Diagramme de séquence : requestAI

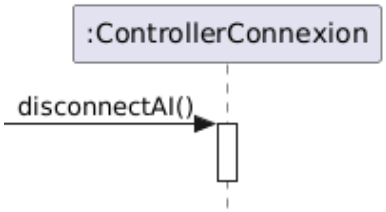


FIGURE 3.4 – Diagramme de séquence : disconnectAI

3.2 Choix techniques

3.2.1 Base de donnée

3.2.1.1 Choix des serveurs

Pour que notre logiciel soit utilisable, il fallait trouver un moyen de stocker les conversions des utilisateurs et de l'IA. Nous avons donc mis en place une base de donnée. Pour l'héberger, nous avons hésité entre deux serveurs, ceux de Supabase et Render.

Render permet de stocker un backend en plus d'une base de donnée mais la version gratuite supprime toute la base au bout d'un mois. De son coté, les serveur de Supabase conserve les données mais ne permettent pas de stocker du backend. Nous avons choisi cette seconde solution. Cependant, si nous devons à terme déployer le logiciel, nous utiliserons aussi les serveurs de Render pour stocker notre backend.

La communication avec les serveurs de Supabse est bien documenté en python. Il existe un module que l'on peut importer pour faciliter les requêtes.

3.2.1.2 Authentification

Pour pouvoir d'identifier les utilisateurs et contrôler l'accès aux données, nous avons du mettre en place un système d'authentification. Notre système est assez classique : chaque utilisateur peut s'identifier avec avec une adresse mail et un mot de passe.

La gestion des mots de passe fait l'objet d'une attention particulière. Pour des raisons de sécurités, on ne peut pas stocker les mots de passes en clair. Ils sont systématiquement hachés avant toute utilisation ou comparaison

Parmi les différents algorithmes de hachage, nous avons choisi Argon2. C'est un algorithme de hachage très efficace ayant gagné la Password Hashing Competition en juillet 2015. Il offre une forte résistance aux attaques par force brute et son implémentation en Python est bien documentée.

FIGURE 3.5 – Argon2

Nous avons ensuite implémenté les fonctions de connexion et création de compte, ce qui nous a donné une première version du système d'authentification.

3.2.2 Interface utilisateur

Notre objectif pour l'interface utilisateur était de proposer une interface aboutie, se rapprochant d'un produit final utilisable, plutôt que d'un simple outil de démonstration avec une interface minimaliste basée sur des solutions comme Streamlit. Nous avons donc choisi de faire une interface web moderne à l'aide du framework React, via Next.js. Cette interface possède pour l'instant 2 pages : la page principale et une d'authentification

3.2.2.1 Page d'authentification

Il s'agit de la première page qu'un utilisateur rencontrera. Lorsque l'on utilise le logiciel, on est automatiquement redirigé vers cette page.

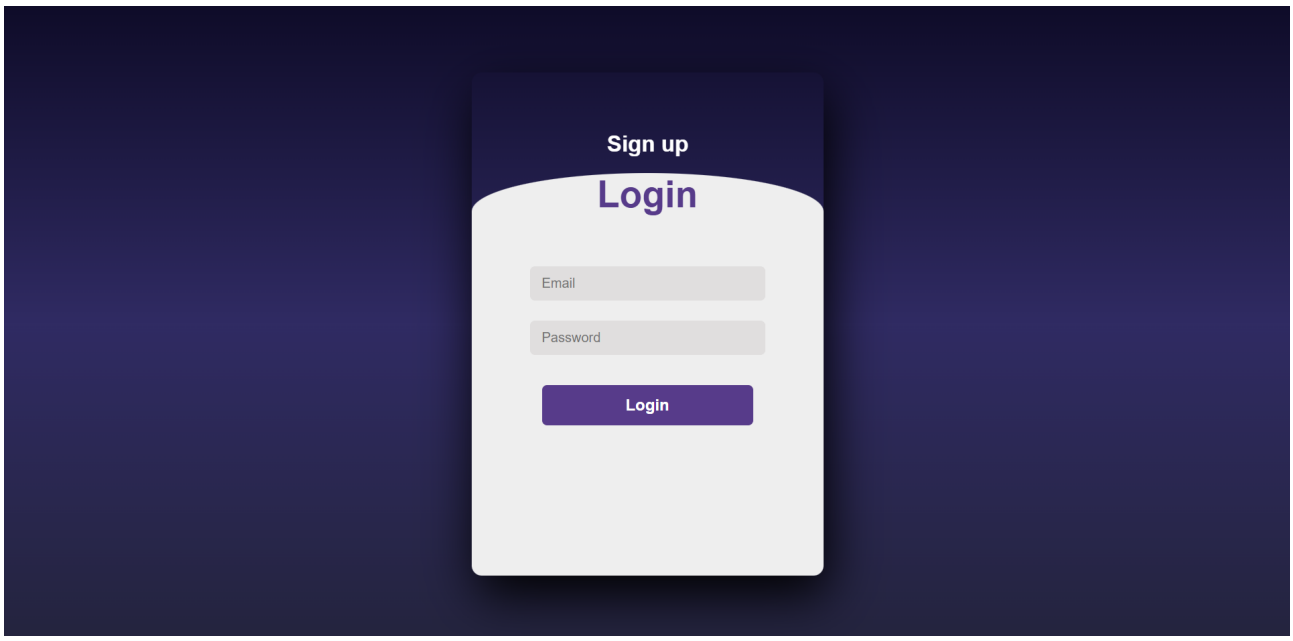


FIGURE 3.6 – Page d'authentification

3.2.2.2 Page principale

Il s'agit de la page sur laquelle on peut communiquer avec l'IA. On s'est inspiré des autres interfaces de chats : un menu sur le coté avec toutes les options et un bloc principal qui sert pour la conversation.

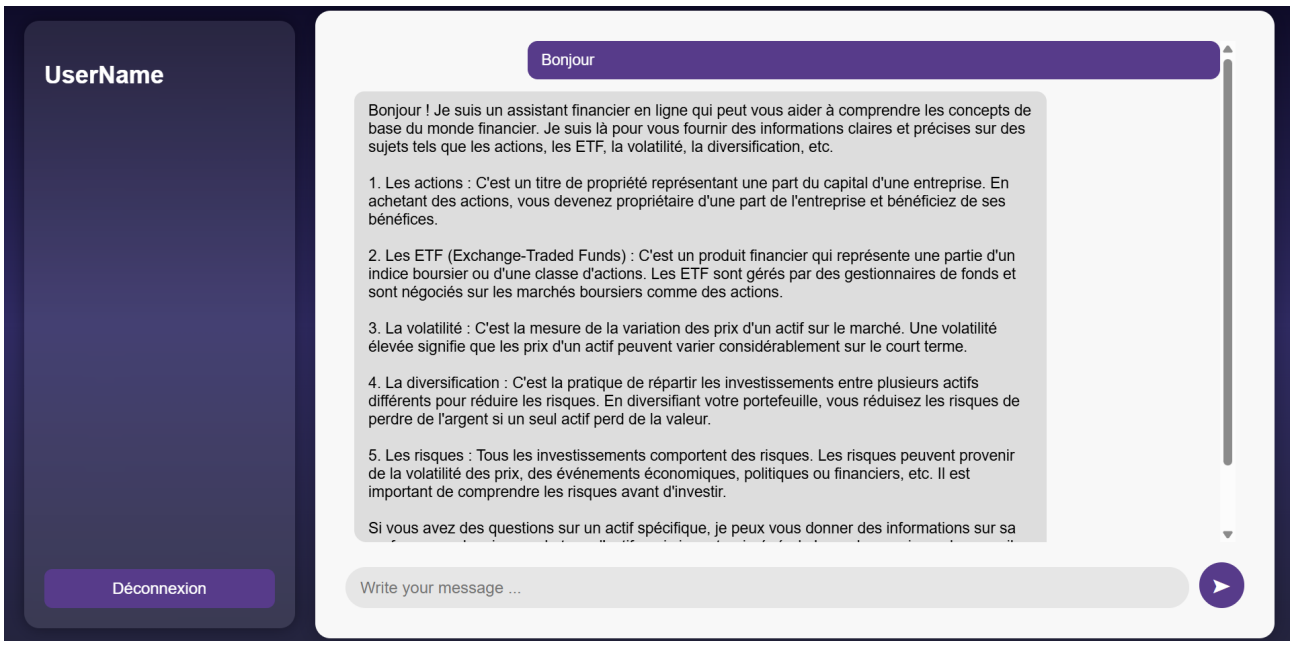


FIGURE 3.7 – Page principale

3.2.2.3 Interaction avec le backend

L'interface utilisateur étant développée en JavaScript et le backend enr Python, nous avons du trouver un moyen pour les faire communiquer.

A l'aide du framework FastAPI, le backend devient une API avec laquelle peut communiquer l'interface utilisateur. Le code est ensuite déployée et exécutée via Uvicorn, un serveur ASGI adapté aux applications asynchrones. La communication entre le frontend et le backend s'effectue par l'intermédiaire de cette API, avec des requêtes.

3.3 Principaux axes d'améliorations

3.3.1 Stockage des conversations

La principale utilité de la base de donnée est de stocker les conversations. Pour des soucis de confidentialité, on ne peut pas les stocker en clair, il faut les chiffrés. Cette partie n'a pas encore été réalisé et demandera de choisir un système de cryptage fiable et efficace.

3.3.2 Base de donnée temporaire

Comme nous utilisons la version gratuite des serveurs de Supabse pour notre base de donnée, en cas d'une semaine d'inactivité la base se ferme.

3.3.3 Déploiement

Si le reste du projet a bien avancé, nous envisageons de déployer le logiciel.

Chapitre 4

Partie IA

4.1 Etat de l'art

L'essor récent des modèles de langage de grande taille (*Large Language Models*, LLM) a profondément transformé le domaine de l'intelligence artificielle, en particulier pour les applications nécessitant des interactions. Entraînés sur de vastes corpus textuels, ces modèles sont capables de comprendre des requêtes complexes, de reformuler des informations et de produire des réponses structurées dans des contextes variés.

Dans le secteur financier, les LLM sont principalement employés comme des outils d'assistance à l'analyse et à la prise de décision. Ils sont notamment utilisés pour l'interprétation de documents financiers, la génération de synthèses, l'explication de concepts économiques complexes ou encore la transformation de requêtes exprimées en langage naturel en instructions exploitables par des systèmes informatiques. Leur capacité à servir d'interface entre des utilisateurs non experts et des outils techniques constitue un levier important pour améliorer l'accessibilité des services financiers.

Néanmoins, l'utilisation des LLM dans un contexte financier présente des limites significatives. Ces modèles ne sont pas conçus pour produire des données chiffrées fiables ou actualisées et peuvent générer des informations incorrectes, un phénomène communément désigné sous le terme d'*hallucination*. Par ailleurs, ils ne disposent pas d'un accès direct aux marchés financiers en temps réel et ne peuvent garantir, à eux seuls, la conformité réglementaire des réponses produites.

Face à ces limitations, les travaux récents privilégient des architectures hybrides intégrant les LLM de manière contrôlée. Dans ce type d'approche, le modèle de langage est utilisé pour analyser et structurer les intentions de l'utilisateur, tandis que l'accès aux données de marché et les calculs financiers sont confiés à des outils spécialisés et déterministes. Cette séparation permet de tirer parti des capacités linguistiques des LLM tout en conservant la rigueur et la fiabilité requises pour des applications financières.

C'est dans cette logique que s'inscrit la partie intelligence artificielle de notre projet. Le modèle de langage n'y est pas employé comme un moteur de décision financière autonome, mais comme un intermédiaire chargé de traduire les besoins exprimés par l'utilisateur en contraintes formelles, ensuite exploitées par les modules financiers présentés dans la suite de ce rapport.

4.2 Choix techniques et architecture de l'agent

Les choix techniques et l'architecture de la partie intelligence artificielle ont été conçus conjointement afin de répondre aux contraintes spécifiques d'un usage financier : fiabilité des résultats, maîtrise des risques d'hallucination et intégration fluide avec les modules logiciel et finance du projet. L'objectif n'est pas de déléguer la décision financière à un modèle de langage, mais de l'utiliser comme une interface intelligente entre l'utilisateur et des outils de calcul déterministes.

4.2.1 Choix du modèle de langage

Parmi les différents modèles de langage disponibles, nous avons retenu un modèle open source de la famille *Mistral*. Ce choix repose sur plusieurs critères : de bonnes performances sur des tâches de compréhension et de génération de texte, une facilité d'intégration technique et l'absence de dépendance à une solution propriétaire.

Ce choix s'inscrit également dans une volonté de privilégier une solution européenne, en cohérence avec le contexte réglementaire et la sensibilité des données financières manipulées dans le cadre du projet. Les paramètres du modèle, notamment une température faible, ont été ajustés afin de favoriser des réponses stables et reproductibles, adaptées à un contexte d'ingénierie.

4.2.2 Orchestration et intégration dans le système

L'intégration du modèle de langage est réalisée à l'aide du framework LangChain, qui permet de structurer les interactions avec le LLM et de l'insérer comme un composant du pipeline applicatif. Le modèle n'est pas utilisé comme un agent autonome capable d'exécuter directement des actions, mais comme un module spécialisé dans l'analyse et la structuration des requêtes utilisateur.

Cette orchestration permet de contrôler précisément les situations dans lesquelles le modèle est sollicité et d'éviter qu'il n'intervienne dans des tâches critiques nécessitant des garanties de fiabilité élevées.

4.2.3 Architecture hybride de l'agent

L'agent repose sur une architecture hybride combinant un modèle de langage et des outils déterministes implémentés en Python. Le rôle du LLM est limité aux tâches de compréhension du langage naturel et de transformation des intentions de l'utilisateur en représentations structurées.

Les opérations nécessitant des données chiffrées ou des calculs financiers — telles que la récupération des données de marché, le calcul des rendements, de la volatilité ou du ratio de Sharpe — sont exclusivement réalisées par des modules déterministes. Cette séparation permet de garantir que les résultats numériques reposent sur des données réelles et des méthodes reproductibles.

4.2.4 Génération de contraintes structurées

L'une des fonctionnalités centrales de l'agent consiste à traduire une demande utilisateur exprimée en langage naturel en un ensemble de contraintes formelles. Ces contraintes sont représentées sous la forme de fichiers JSON respectant un schéma prédéfini, commun avec le module d'optimisation financière.

Cette structuration assure une interface claire et robuste entre la partie IA et la partie finance du projet. Elle permet également de valider les sorties du modèle de langage en restreignant les champs, opérateurs et valeurs autorisés, réduisant ainsi les risques d'incohérences ou d'interprétations erronées.

4.2.5 Flux global de fonctionnement

Le fonctionnement global de l'agent peut être résumé par le flux suivant :

- L'utilisateur formule une requête en langage naturel via l'interface.
- Le système analyse la nature de la requête afin de déterminer si elle relève d'une structuration sémantique ou d'un calcul financier.
- Le cas échéant, le modèle de langage génère une représentation structurée de la demande sous forme de contraintes formelles.
- Les modules financiers exploitent ces contraintes ou réalisent les calculs nécessaires.
- Les résultats sont restitués à l'utilisateur sous une forme interprétable.

Cette architecture garantit une séparation claire des responsabilités entre les composants et constitue un élément clé pour assurer la robustesse et la fiabilité du système.

4.3 Pipeline de traitement et garde-fous

Le pipeline de traitement de la partie intelligence artificielle a été conçu afin de garantir à la fois la fluidité de l'interaction avec l'utilisateur et la fiabilité des résultats produits. Une attention particulière a été portée à la mise en place de garde-fous visant à limiter les erreurs, les incohérences et les risques liés à l'utilisation d'un modèle de langage dans un contexte financier.

4.3.1 Pipeline de traitement

Le traitement d'une requête utilisateur suit une chaîne d'étapes bien définie. Dans un premier temps, la requête exprimée en langage naturel est analysée afin d'identifier son type : demande informative, requête nécessitant des calculs financiers ou formulation de contraintes pour la construction d'un univers d'investissement.

Lorsque la requête implique une structuration sémantique, le modèle de langage est sollicité pour transformer la demande en une représentation formelle standardisée. Cette représentation prend la forme d'un ensemble de contraintes exprimées dans un format JSON, compatible avec le module d'optimisation financière présenté dans le chapitre Finance.

Dans le cas des requêtes nécessitant des données chiffrées, le pipeline contourne le modèle de langage et fait directement appel aux outils de calcul déterministes. Cette organisation permet d'adapter dynamiquement le chemin de traitement en fonction de la nature de la demande.

4.3.2 Mise en place de garde-fous

L'utilisation d'un modèle de langage dans un cadre financier impose la mise en place de mécanismes de contrôle stricts. Plusieurs garde-fous ont ainsi été intégrés au pipeline.

Tout d'abord, les sorties générées par le modèle de langage sont systématiquement validées avant d'être exploitées par les autres modules. Les champs, opérateurs et valeurs autorisés sont limités à un ensemble prédéfini, ce qui empêche l'introduction de contraintes incohérentes ou non supportées par le système.

Ensuite, les calculs financiers et l'accès aux données de marché sont exclusivement réalisés par des modules déterministes, indépendants du modèle de langage. Cette séparation réduit fortement les risques d'hallucination et garantit la reproductibilité des résultats.

Enfin, le rôle du modèle de langage est volontairement restreint à des tâches d'assistance et de structuration. Il ne produit ni recommandations financières directes, ni décisions d'investissement autonomes, ce qui permet de conserver un cadre d'utilisation maîtrisé et conforme aux objectifs du projet.

4.4 Exemples d'utilisation

Cette section présente plusieurs exemples concrets d'utilisation de l'agent d'intelligence artificielle, illustrant les principaux cas d'usage du système : la structuration de demandes d'investissement exprimées en langage naturel et le calcul d'indicateurs financiers à partir de données de marché.

4.4.1 Structuration d'une demande en contraintes formelles

L'utilisateur peut formuler une demande décrivant un univers d'investissement en langage naturel. Par exemple, la commande suivante est soumise à l'agent :

```
screener actions françaises du secteur énergie, capitalisation > 1 milliard, dividende
positif
```

Le modèle de langage analyse cette requête et en extrait les contraintes implicites. Il génère alors une représentation structurée sous forme d'un fichier JSON respectant un schéma prédéfini, compatible avec le module d'optimisation financière. Le résultat obtenu est le suivant :

Code 4.1 – Contraintes générées par l'agent IA pour un univers d'actions françaises

```
{
  "universe": "equity",
  "constraints": [
    {
      "field": "sector",
      "operator": "is-in",
      "values": ["Energy"]
    },
    {
      "field": "lastclosemarketcap.lasttwelvemonths",
      "operator": "gt",
      "values": [1000000000]
    },
    {
      "field": "forward_dividend_yield",
      "operator": "gt",
      "values": [0]
    },
    {
      "field": "region",
      "operator": "is-in",
      "values": ["fr"]
    }
  ],
  "notes": ""
}
```

Cette structuration permet de garantir une interface robuste entre la partie IA et la partie finance du projet. Les contraintes générées peuvent être directement exploitées pour filtrer l'univers d'investissement avant l'étape d'optimisation de portefeuille.

4.4.2 Structuration d'une contrainte de risque implicite

L'agent est également capable d'interpréter des notions de risque formulées de manière qualitative par l'utilisateur. Par exemple, la commande suivante est exécutée :

```
screener Actions Europe, secteur Santé, volatilité faible (beta < 1)
```

Dans ce cas, la notion de *faible volatilité* est interprétée par l'agent à l'aide d'un indicateur financier proxy, le bêta de l'actif. Le modèle de langage génère alors les contraintes suivantes :

Code 4.2 – Contraintes générées pour un univers d'actions européennes à faible volatilité

```
{
  "universe": "equity",
  "constraints": [
    {
      "field": "sector",
      "operator": "is-in",
      "values": ["Healthcare"]
    },
    {
      "field": "beta",
      "operator": "lt",
      "values": [1]
    },
    {
      "field": "region",
      "operator": "is-in",
      "values": [
        "at","be","ch","cz","de","dk","ee","es","fi","fr",
        "gb","gr","hu","ie","it","lt","lv","nl","no","pl",
        "pt","ro","se"
      ]
    }
  ],
  "notes": "La volatilit  est ici approch e par le b ta , utilis  comme proxy du risque
      syst matique."
}
```

Cet exemple met en évidence la capacité de l'agent à traduire des concepts financiers qualitatifs en contraintes quantitatives exploitables, tout en explicitant les hypothèses retenues lorsque certaines informations ne sont pas directement filtrables.

4.4.3 Comparaison quantitative de deux actifs

Dans le cas d'une requête portant sur des indicateurs financiers chiffrés, l'agent n'utilise pas le modèle de langage pour produire les valeurs numériques. Par exemple, la commande suivante est exécutée :

```
compare ORA.PA EN.PA 6mo
```

Cette commande déclenche la récupération des données de marché via l'API Yahoo Finance, puis le calcul déterministe des indicateurs financiers sur une période de six mois. Les résultats obtenus sont les suivants :

- **ORA.PA (Orange)**
  - Rendement annualisé $\approx 23.28\%$
  - Volatilité annualisée $\approx 16.69\%$
  - Ratio de Sharpe annualisé ≈ 1.14
- **EN.PA (Bouygues)**
  - Rendement annualisé $\approx 33.45\%$
  - Volatilité annualisée $\approx 22.31\%$
  - Ratio de Sharpe annualisé ≈ 1.21

Cet exemple illustre la séparation stricte entre raisonnement linguistique et calcul quantitatif mise en place dans notre architecture. Les indicateurs financiers sont calculés à partir de données réelles et de méthodes déterministes, ce qui garantit la fiabilité et la reproductibilité des résultats.

Chapitre 5

Partie Finance

5.1 Introduction

L'objectif de ce chapitre est de comprendre comment nous allons pouvoir construire un portefeuille "efficace" selon un ensemble de contraintes voulues par le client.

Pour atteindre cet objectif, nous devons construire une stratégie d'investissement. Un portefeuille peut contenir de nombreux produits financiers : des actions (titres d'entreprises), des obligations (créances d'entreprises ou d'états), des fonds (panier de produits financiers ayant des propriétés très variables), des options. les options et obligations ne sont pas aisés à traiter sur les marchés, les informations à leurs sujets ainsi que les plateformes d'échanges sont moins démocratisées que pour les actions et les fonds. Dans un soucis d'accessibilité des données (gratuitement) nous allons nous atteler à construire des portefeuilles constitués d'actions et de fonds uniquement.

Dans notre solution la dimension financière est utilisée comme un outil. cet outil est donc indépendant du reste de la solution. Il a donc sa propre architecture et pipeline.

5.2 Etat de l'art

5.2.1 Modèle d'évaluation des actifs financiers

L'optimisation de portefeuille actions n'est pas une science exacte mais une approche consiste à maximiser la rentabilité de ce dernier pour un niveau de risque consentit. Cette approche appelée le Modèle d'évaluation des actifs financiers [3] (MEDAF) a été développée conjointement par Markowitz, Miller et Sharpe en 1990.

Le modèle d'évaluation repose sur les hypothèses suivantes :

- Aucun coût de transaction, d'impôts, vente à découvert possible.
- Existence d'un actif sans risque.
- Tous les actifs sur le marché sont disponibles à l'achat.
- Les marchés sont efficients.
- Les anticipations sont homogènes.
- Tous les investisseurs ont le même horizon d'investissement.
- Tous les agents sont Price-takers.

A l'équilibre du marché, il existe une relation linéaire entre la rentabilité espérée de notre portefeuille et son risque systématique :

$$E(\tilde{r_i}) = r_f + \beta_i(E(\tilde{r_M}) - r_f) \tag{5.1}$$

Dans cette équation r_i représente la rentabilité implicite de l'actif i, r_f représente la rentabilité de l'actif sans risque et r_M représente la rentabilité implicite observée ou attendue pour tout le marché.

Le β_i représente quant à lui le risque systématique de l'actif i [1]. Le bêta est un indice de comparaison de l'élasticité d'un actif face aux variations du marché. Il est calculé grâce à l'équation suivante :

$$\beta_i = \frac{Cov(r_i, r_M)}{Var(r_M)} \tag{5.2}$$

5.2.2 Optimisation sous contraintes

Un portefeuille P est composé de n produits financiers distincts chacun présent dans une proportion w_i. Notre portefeuille P ne contiendra pas de titre à découvert donc

$$\forall i \in [1, n], w_i >= 0 \tag{5.3}$$

$$\sum_{k=1}^{n} w_i = 1 \tag{5.4}$$

Pour constituer notre portefeuille nous devons allouer des poids w_i à chacun de nos n actifs. Nous allons devoir trouver une mesure de "performance" du portefeuille.

Étant donné que nous souhaitons construire des portefeuille avec un horizon moyen long terme, il est important d'avoir un bon équilibre entre le rendement du portefeuille et le risque pris par ce dernier.

Une mesure communément utilisée est le ratio de Sharpe. Il mesure l'écart de performance de notre portefeuille P avec le taux sans risque divisé par la volatilité de ce portefeuille P. On peut le résumer dans la formule suivante :

$$S = \frac{r_P - r_f}{\sigma_P} \tag{5.5}$$

Ce ratio de Sharpe sera donc notre fonction à maximiser, on peut le réécrire en fonction des poids de notre portefeuille P.

$$S(w_1, ..., w_n) = \frac{\sum_{k=1}^{n} w_i r_i - r_f}{\sqrt{\sum_{k=1}^{n} w_i^2 \sigma_i^2 + \sum_{k=1, k\neq j}^{n} \sum_{j=1, j\neq k}^{n} w_i w_j Cov(r_i, r_j)}} \tag{5.6}$$

Cette fonction à n arguments en a en réalité $n - 1$ du fait de la contrainte sur la somme des poids.

On construit les valeurs r_i et σ_i sur les valeurs historiques prises par les actifs considérés. Au vu de la période d'investissement minimale de nos portefeuilles nous regardons les historiques de prix sur 6 mois. Cette construction de portefeuille par les performances passées pose un problème structurel et nous y reviendrons dans la partie portant sur les limites de cette méthode.

Sur cette période d'observation nous relevons les prix de l'actif i chaque jour j d'ouverture du marché à la fermeture pendant m jours : p_{ij}. A partir de ce vecteur P_i des prix, on peut construire le vecteur de rentabilité de l'actif pour chaque jour du 2e au m^{ime} jour.

$$R_i = \begin{pmatrix} \frac{p_{i2} - p_{i1}}{p_i 1} \\ \vdots \\ \frac{p_{im} - p_{i(m-1)}}{p_i (m-1)} \end{pmatrix} \tag{5.7}$$

Nous calculons ensuite la matrice de covariance entre les rentabilités R_i.

$$Cov(R_i, R_k) = \frac{1}{m-1} \sum_{j=1}^{m-1} (R_{ij} - \bar{R_i})(R_{kj} - \bar{R_k}) \tag{5.8}$$

5.2.3 Exemple d'optimisation à deux actifs

Nous avons désormais tous les éléments pour construire notre premier portefeuilles à 2 actifs.

Prenons par exemple deux entreprises du CAC40 : Orange (ORA.PA) et Bouygues SA (EN.PA).
Nous récupérons les historiques de prix sur une période de 6 mois pour les deux titres et nous calculons la matrice de rentabilité ainsi que la matrice de covariance entre les deux actifs.

Nous pouvons écrire le ratio de Sharpe comme :

$$S(w_1, w_2) = \frac{w_1 r_1 + w_2 r_2 - r_f}{\sqrt{w_1^2 \sigma_1^2 + w_2^2 \sigma_2^2 + 2 w_1 w_2 Cov(r_1, r_2)}} \tag{5.9}$$

Or $w_2 = 1 - w_1$. Donc notre fonction à maximiser 5.9 se réécrit comme :

$$S(w_1) = \frac{w_1 r_1 + (1 - w_1) r_2 - r_f}{\sqrt{w_1^2 \sigma_1^2 + (1 - w_1)^2 \sigma_2^2 + 2 w_1 (1 - w_1) Cov(r_1, r_2)}} \tag{5.10}$$

On peut construire le graphique représentant le ratio de Sharpe du portefeuille en fonction du point d'un actif (le poids du second en découle).

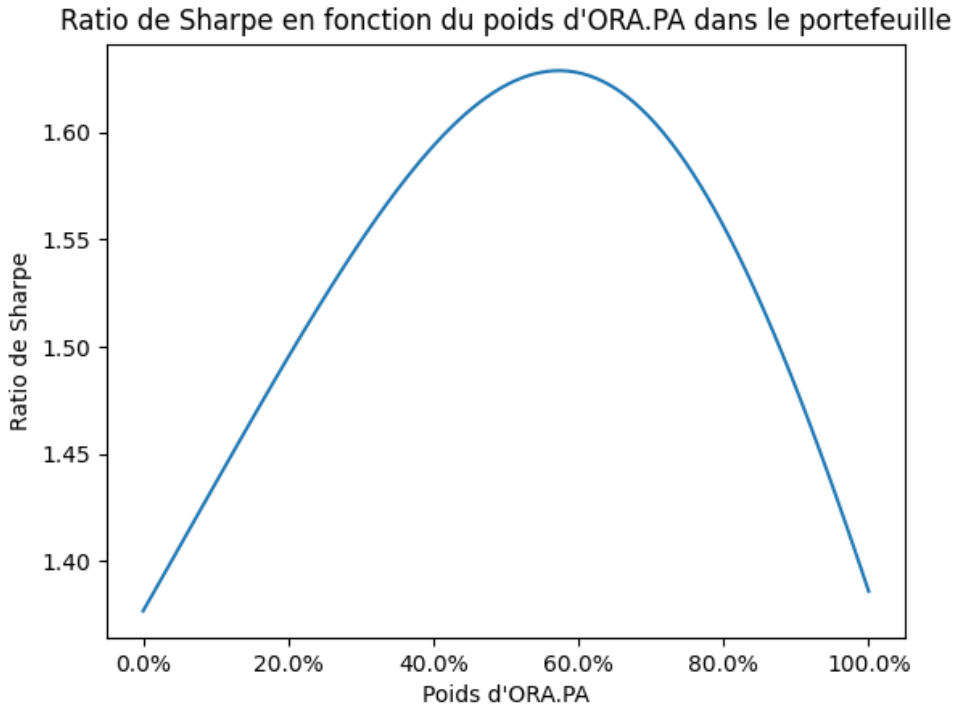


FIGURE 5.1 – Ratio de Sharpe en fonction du poids d'ORA.PA dans le portefeuille

La représentation graphique de la courbe de Sharpe en fonction du poids w_1 est concave et admet un maximum en 0.573. Donc un portefeuille optimal comportant 57.3% d'actions orange (ORA.PA) et 42.7% d'actions Bouygues (EN.PA).
Dans notre outil de construction de portefeuille nous procédons de manière analogue en dimension n.

5.3 Architecture

La structure indépendante du module d'optimisation du portefeuille permet une grande flexibilité. Pour cela nous avons besoins de plusieurs sous-modules :

— Un sous-module pour construire un univers d'investissement action.
— Un sous-module pour optimiser un portefeuille.

Le sous-module de construction de l'univers d'investissement repose sur un ensemble de critères sur les actions choisis par le client. L'ensemble de ces critères sont exprimés au travers de l'interface utilisateur, ce demande est structurée afin d'obtenir un format prédéfini. Nous abordons le sujet de constructions de ces filtres dans le chapitre IA du rapport. Nous recevons ainsi un ensemble de critères standardisés basés sur les services disponibles dans l'API que nous requêtons.
Voici un exemple de fichier JSON accepté.

Code 5.1 – Contraintes d'investissement transmises au module d'optimisation

```
{
  "universe": "equity",
  "constraints": [
    {
      "field": "region",
      "operator": "is-in",
      "values": [
        "fr"
      ]
    },
    {
      "field": "sector",
      "operator": "is-in",
      "values": [
        "Energy"
      ]
    }]
}
```

Chaque contrainte doit présenter au minimum 3 champs d'information : un nom (field), un opérateur (operator) et une liste de valeurs (values).
Concrètement nous avons construit une classe EquityScreener ainsi qu'une sous classe Filter qui nous permet de structurer l'univers d'investissement. Ensuite nous utilisons une classe Portfolio qui a pour but de construire le portefeuille final.

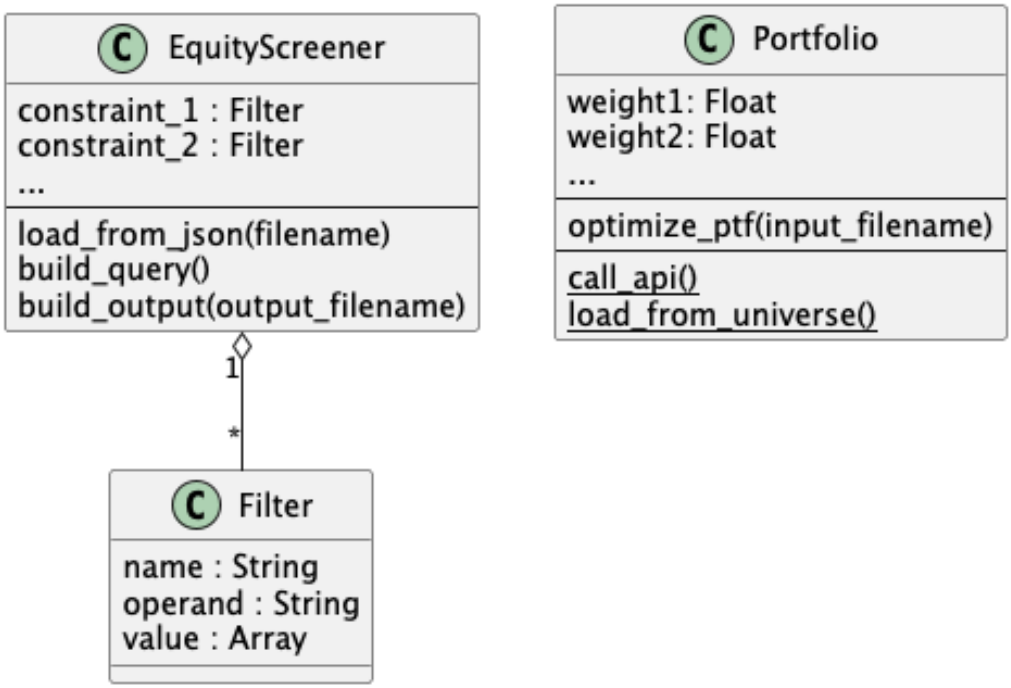


FIGURE 5.2 – Diagramme de classes du module d'optimisation de portefeuille

5.4 Choix techniques

5.4.1 Sources de données

Il nous faut donc connaître tous les produits financiers qui nous sont investissables ainsi que des caractéristiques sur ces derniers pour pouvoir les filtrer. L'entreprise Accenture nous a spécifié que l'ensemble de notre projet devait être fonctionnel gratuitement. Ceci implique un certain nombre de restriction surtout dans le domaine des api fournissant des données de marchés. Beaucoup de ces dernières fonctionnent avec un système d'abonnement. Une des concession à faire pour une utilisation gratuite de ces APIs est de ne pas bénéficier des data en temps réel avec avec un délai par rapport à la réalité (souvent autour des 24h). Ceci n'est cependant pas limitant pour notre approche étant donné que nous souhaitons construire des stratégies d'investissement à moyen long terme. Nous ne sommes ainsi pas à 24h près. Nous avons finalement le choix entre 3 sources d'informations :

— Alpha Vantage : la plateforme ne propose pas d'intégration directe, l'utilisation se fait via des appels API. il y a une limite d'utilisation journalière. Cependant elle offre un plus large panel d'informations et de supports (comparé à Yahoo Finance)
— Yahoo Finance : la plateforme est entièrement gratuite, la limite est situé sur la taille des requêtes mais par de limite de fréquence, donc le découpage en plusieurs requêtes est possible. Il a été développé une librairie Python pour faciliter son utilisation, la documentation est complète. La limite principale est l'absence d'informations "intraday" et le faible nombre d'actifs financiers présents (actions et quelques fonds).
— Twelve Data : le compte gratuit limite à 800 requêtes journalières (peu propice à la scalabilité).

Pour la première partie du projet nous avons donc décidé d'utiliser Yahoo Finance via la librairie développée par Ranaroussi [4] pour une utilisation directement dans python de l'API de Yahoo qui elle n'est pas officiellement documentée mais accessible.
Dans un second temps nous pourrions proposer une extension de l'outil à d'autres classse d'actifs, dans ce cas les services proposés par Yahoo Finance ne seront plus suffisant et il faudra envisager de basculer sur une solution comme Alpha Vantage plus complète.

5.4.2 ajustement du portefeuille

A ce stade du projet du sommes en mesure de construire un portefeuille basé sur univers d'investissement défini par le client. Nous souhaitons proposer dans la seconde moitié du projet une fonctionnalité de rebalancement du portefeuille en fonction des mouvements de marché pour maximiser la performance de ce dernier. Cette fonctionnalité nécessitera la mise en production de la base de données avec les informations portefeuilles des clients. Il nous faudra également mener des recherches pour définir les indicateurs qui déclencheront la réévaluation des poids, l'ajout ou la suppression d'actifs.

5.5 Problèmes rencontrés

5.5.1 Disponibilité d'informations financières

L'information est la clé de voûte des marchés financiers et par conséquence, La détention d'informations pertinentes représente une opportunité immédiate. Une hypothèses de beaucoup de concepts financiers repose sur l'efficience des marchés et l'absence d'opportunité d'arbitrage. L'efficience des marchés financiers suggère que l'ensemble des informations disponibles sont contenues dans le prix d'un actif. Dans notre projet nous avons également fait cette hypothèse car nous construisons les portefeuilles uniquement grâces aux valeurs historiques de l'actif. L'absence d'opportunité d'arbitrage signifie qu'il n'est pas possible de réaliser un gain immédiat et sans risque en achetant un actif et revendant un autre actif. Dans les faits les opportunités d'arbitrages existent mais elles sont négligeables car elles sont traitées très rapidement par une catégorie particulière de traders : les arbitragistes. Ces derniers utilisent des algorithmes pour détecter les opportunités d'arbitrage et réalisent des profits en rééquilibrant le marché.

Ces deux points devraient nous assurer qu'en étudiant les prix des actifs dans lequels on souhaite investir on est en mesure d'analyser tout le potentiel de ces actifs et on n'a de toute façon pas de moyen de réaliser un profit sans risque, alors le profit sera théoriquement proportionnel au risque prix (comme nous l'avons vu dans le MEDAF).

Une conséquence de ces hypothèses est que les données de marchés ont beaucoup de valeurs et les distributeur d'informations le savent bien. Si on peut trouver librement les valeurs journalières de beaucoup d'actifs simples (actions, fonds, obligations). Il peut devenir très coûteux d'accéder en direct au cours d'une action. Les fonds ne fournissent pas de valeurs en temps réel. A titre d'exemple un terminal Bloomberg (le fournisseur d'information financière leader du marché) coûte environ 30000$ par an et par utilisateur.

Pour notre projet les horizons d'investissement sont long et donc nous n'aurons pas besoin d'une telle précision dans l'information, ce qui nous donne l'espoir de trouver des sources d'informations gratuites.

5.5.2 Construction de portefeuilles sur des données historiques

Comme évoqué dans la section précédente, nous construisons des portefeuilles à l'aide des données de prix historiques. Cette méthode est censé être la meilleure sous l'hypothèse de marchés financiers efficients (ce qui reste à démontrer). Elle pose cependant un problème de taille. Nous construisons le portefeuille le plus efficient sur une période passée (6 mois dans notre cas), c'est à dire le portefeuille qui a le meilleur rapport bénéfice

risque. Néanmoins rien ne nous assure que sur la période à venir ce soit toujours le cas. Une solution partielle à ce problème est de vérifier régulièrement que l'efficience du portefeuille prédite est bien ce qu'il se passe en réalité et réajuster ce dernier. Si la performance de ce dernier n'est pas satisfaisante un rééquilibrage permettra d'augmenter la performance ou à minima de limiter la perte. A l'inverse si la performance est satisfaisante alors le portefeuille sera laissé intact. Ce sera l'objet de nos réflexions pour la seconde partie du projet. Ce nouveau sujet introduira une nouvelle question : Quelle est la période idéale pour réévaluer un portefeuille ? Faut-il combiner cette période à des indicateurs ?

Chapitre 6

Conclusion

Ce projet de fin d'études s'inscrit dans un contexte de transformation profonde des services financiers, marqué par l'essor des modèles de langage et leur intégration croissante dans les outils d'aide à la décision. L'objectif était de concevoir un cadre applicatif permettant de tirer parti des capacités des LLM tout en répondant aux exigences de fiabilité, de transparence et de maîtrise des risques propres au domaine financier.

Les travaux réalisés à mi-parcours ont permis de poser des bases solides, tant sur le plan conceptuel que technique. Une architecture en trois couches — logicielle, intelligence artificielle et finance — a été définie et implémentée, garantissant une séparation claire des responsabilités. Le modèle de langage est utilisé comme une interface de structuration des besoins exprimés en langage naturel, tandis que les calculs financiers et l'optimisation de portefeuilles reposent sur des modules déterministes, assurant la reproductibilité et la robustesse des résultats.

Les premiers développements ont permis de valider la pertinence de cette approche hybride, notamment à travers la structuration de contraintes d'investissement, la comparaison quantitative d'actifs et la construction de portefeuilles sous contraintes. Ces résultats démontrent la capacité du système à répondre de manière cohérente à des demandes complexes, tout en conservant un cadre d'utilisation maîtrisé.

La suite du projet portera sur l'enrichissement fonctionnel et la consolidation du système. Les principaux axes d'amélioration concernent le déploiement de l'application, la gestion sécurisée et persistante des données, l'extension à de nouvelles classes d'actifs ainsi que l'introduction de mécanismes de rééquilibrage dynamique des portefeuilles. Ces évolutions permettront de rapprocher davantage la solution développée d'un produit utilisable dans un contexte réel, tout en approfondissant les enjeux techniques et financiers abordés dans ce projet.

Bibliographie

[1] *Coefficient bêta*. URL : https://fr.wikipedia.org/wiki/Coefficient_b%C3%AAta (visité le 04/01/2026).

[2] Laurence DAURES. "Cours fondamental de finance, ESSEC". In : ().

[3] Frank FABOZZI, Harry MARKOWITZ et Francis GUPTA. "Portfolio Selection". In : sept. 2008. ISBN : 9780470404324. DOI : 10.1002/9780470404324.hof002001.

[4] RANAROUSSI. *Documentation yfinance*. URL : https://ranaroussi.github.io/yfinance/advanced/index.html (visité le 04/01/2026).

Annexes

FIGURE 6.1 – Cas d'utilisation : création de compte

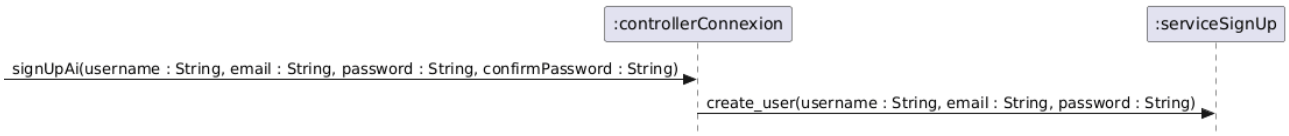


FIGURE 6.2 – Diagramme de séquence : signUpAI

FIGURE 6.3 – Cas d'utilisation : Charger une conversation

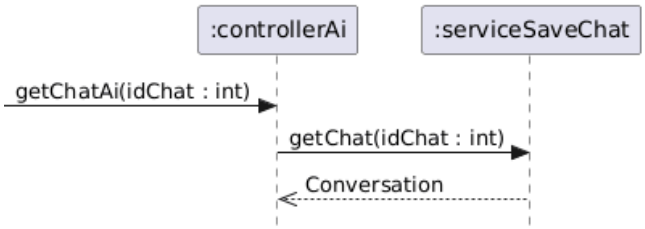


FIGURE 6.4 – Diagramme de séquence : getChatAI